

# TicTacToe Arena — Project Report

## Table of Contents

|                                                     |    |
|-----------------------------------------------------|----|
| Executive Summary .....                             | 2  |
| 1. Introduction .....                               | 2  |
| Goals: .....                                        | 2  |
| Assumptions / Unknowns (explicit):.....             | 3  |
| 2. Business Objectives & Success Metrics .....      | 3  |
| 3. Functional Requirements .....                    | 3  |
| 4. Non-Functional Requirements.....                 | 4  |
| 5. Technical Architecture .....                     | 5  |
| High-Level Design (HLD) .....                       | 5  |
| Tech Stack.....                                     | 5  |
| Data Flow.....                                      | 6  |
| Event & Payload Conventions .....                   | 6  |
| 6. System Components (Detailed) .....               | 6  |
| 7. Data Model (High-level) .....                    | 6  |
| 8. API Contracts & Endpoints (Representative) ..... | 7  |
| 9. Distributed Systems & Scalability Strategy ..... | 7  |
| Current design strengths .....                      | 7  |
| Recommendations .....                               | 7  |
| 10. Cybersecurity & Compliance .....                | 7  |
| Authentication & Authorization .....                | 7  |
| Threat analysis (high level) .....                  | 8  |
| Data protection & compliance .....                  | 8  |
| Concrete actionable security items .....            | 8  |
| 11. Testing Strategy .....                          | 8  |
| 12. Implementation Roadmap (Phased timeline) .....  | 9  |
| 13. Risk Assessment .....                           | 9  |
| 14. Operational Runbook .....                       | 10 |
| 15. Deployment & CI/CD (Recommendations).....       | 11 |
| 16. Cost & Capacity Estimates.....                  | 11 |
| 17. Conclusion .....                                | 11 |

|     |                  |    |
|-----|------------------|----|
| 18. | References ..... | 12 |
| 19. | Glossary.....    | 13 |
| 20. | Appendices ..... | 17 |

## Executive Summary

TicTacToe Arena is a low-latency, multiplayer game platform designed as a collection of small, focused Flask microservices and a Vite/React frontend. The system's runtime design prioritizes responsiveness (Redis-backed runtime state), decoupling (Redis Pub/Sub event bus), and developer ergonomics (per-service Dockerfiles and simple local verifier scripts). Immediate priorities are:

- 1) Fix observed frontend bugs (lobby static names, timer and game-end handling).
- 2) Stabilize event schema contracts and tests.
- 3) Harden authentication flows and Redis availability for production.

Business value: fast turn-based multiplayer with a small operational footprint, enabling rapid experimentation, leaderboards and social features with modest infra costs when scaled prudently.

Key recommendations (top-line):

- Fix front-to-back event contracts and publish an event schema document before cross-service releases.
- Treat Redis as a critical, highly-available component (cluster + replication) and implement write-back reconciliation to the DB.
- Harden cookie-based JWT usage with CSRF protections and short TTLs.

## 1. Introduction

TicTacToe Arena is an online, multiplayer tic-tac-toe platform built as a microservices system. Which delivers a production-capable distributed backend and a responsive frontend supporting matchmaking, real-time gameplay, persistent game records, leaderboards, and user profiles.

Key deliverables: functioning Auth, User Profile, Game, Matchmaking, Leaderboard services, a WebSocket gateway, and a Vite/React frontend.

Goals:

- Low-latency real-time gameplay with per-user Socket.IO rooms.
- Redis-first runtime state for active games; PostgreSQL for durable metadata.

- Decoupled services communicating via Redis Pub/Sub.
- Containerized development/deployment (per-service Dockerfile + docker-compose.yml).

## Assumptions / Unknowns (explicit):

- Production orchestration (Docker Compose + Cloud).
- Expected peak concurrency (active games/sockets).
- Exact authServices token storage and claims: assume JWTs with short-lived access + refresh tokens stored in cookies (user notes request cookies).

## 2. Business Objectives & Success Metrics

- Objective: Deliver a responsive, engaging multiplayer experience for casual competition.
- Metrics:
  - Player retention (7-day retention)
  - Match completion rate (> 99% under normal load)
  - Median move latency < 200ms
  - System availability 99.9% for gateway and game services
  - ELO ranking accuracy (post-match reconciliation errors < 0.1%)

## 3. Functional Requirements

### FR-01 - User Authentication:

- Register, login, logout, password reset, email verification.
- JWT-based tokens; access token short-lived; refresh token persisted securely (cookie).

### FR-02 - User Profiles:

- CRUD operations for user profile (wins, losses, ELO, display name, avatars, ranks).
- Profile queries: list, search, recent/leaderboard-sorted fetch.

### FR-03 - Matchmaking:

- ELO-aware matchmaking with configurable search window and re-check interval.
- Publish match found events to both users.

### FR-04 - Game Lifecycle:

- Create game, initialize runtime state in Redis, apply moves atomically, evaluate win/draw/invalid.
- Publish game\_update, game\_over, and domain GAME\_COMPLETED events.

- Persist finished game metadata to Postgres.

#### FR-05 - Real-Time Transport:

- WebSocket Gateway subscribes to Redis channels and emits to Socket.IO rooms.
- Per-user and per-game room semantics; guarantee ordered delivery per game session.

#### FR-06 - Leaderboard & Stats:

- Consume domain events to update leaderboards and user statistics.
- Provide leaderboard queries with pagination and season support.

#### FR-07 - Admin & Observability:

- Health-check endpoints for each service.
- Verifier script “python be/verify\_all\_services.py” to check local stack.

#### FR-08 - Security & Tokens:

- Cookie HttpOnly + Secure in production
- CSRF protection for cookie flows and origin checks.

## 4. Non-Functional Requirements

#### NFR-01 – Availability:

- Target: 99.9% service availability for gateway and game services in production.

#### NFR-02 – Latency:

- End-to-end move propagation latency < 200ms under normal load.

#### NFR-03 – Scalability:

- Support horizontal scaling: stateless REST services.
- Multiple gateway instances with Redis message queue.

#### NFR-04 - Consistency & Durability:

- Runtime consistency: Redis as source-of-truth for active games.
- Durability: game completion persisted to PostgreSQL database and use reconcilers for missed writes.

#### NFR-05 – Security:

- TLS for all external and inter-service communication in production.
- Redis and PostgreSQL restricted to private networks, with authentication and TLS.

NFR-06 - Observability:

- Metrics (Prometheus): socket connections, Redis ops/sec, matchmaking queue length, move apply latency.
- Structured logs & no PII logged.

NFR-07 – Testability:

- Unit tests for game state transitions.
- Integration tests for Pub/Sub flows and WebSocket Gateway.

NFR-08 - Maintainability:

- Clear per-service structure (config.py, extensions.py, routes.py), documented be/.env.example.

## 5. Technical Architecture

### High-Level Design (HLD)

- Microservices (Flask) each own a bounded context and DB (Postgres) for durable data.
- Redis serves dual roles: runtime state store for gameServices and Pub/Sub message bus for domain and UI events.
- WebSocket Gateway subscribes to Redis channels (game\_updates, domain\_events, notifications, matchmaking\_updates) and forwards messages to clients via Socket.IO rooms.
- Frontend (Vite + React) uses Socket.IO and REST APIs to participate in matchmaking and gameplay.

### Tech Stack

| Layer            | Technology (observed / recommended)                                            |
|------------------|--------------------------------------------------------------------------------|
| Frontend         | Vite, React, Redux Toolkit, TailwindCSS (see fe/README.md)                     |
| Backend          | Python Flask (per-service under be/services/*)                                 |
| Real-time bus    | Redis Pub/Sub (REDIS_URL env)                                                  |
| Runtime state    | Redis (game state under be/services/gameServices/state/)                       |
| DB               | PostgreSQL (service metadata)                                                  |
| WebSockets       | flask_socketio (gateway message_queue=Config.REDIS_URL)                        |
| Containerization | Docker + service docker-compose.yml                                            |
| Background jobs  | Optional Celery (env present) or in-service threads (e.g., timeout_manager.py) |

## Data Flow

1. Client requests matchmaking → matchMakingServices finds/creates a match → requests gameServices to create a game.
2. gameServices creates DB record, initializes Redis state, and publishes game\_start and per-user dashboard\_update events to game\_updates channel with room fields.
3. WebSocket Gateway receives Redis messages and emits Socket.IO events to the appropriate rooms.
4. Moves submitted to gameServices (REST /games/<id>/move) are applied in Redis via apply\_move. If completed, events game\_over and domain GAME\_COMPLETED (on domain\_events) are published.
5. userProfileServices and leaderboardServices consume domain\_events to update stats and leaderboards; frontend receives updates via WebSocket Gateway.

## Event & Payload Conventions

- Pub/Sub message JSON schema: { "event": "<name>", "data": {...}, "room": "<room\_id>" }.
- Domain events: { "event\_type": "<TYPE>", "payload": {...} } on domain\_events.

## 6. System Components (Detailed)

- Auth Service (be/services/authServices): issues and validates JWTs, supports registration, login, logout, password reset, and email verification. Implementation details for cookie storage should be confirmed and hardened.
- Game Service (be/services/gameServices): primary runtime logic like creates games, initializes Redis state (see state/), applies moves (apply\_move) and publishes game\_update/game\_over events.
- WebSocket Gateway (be/services/websocketGateway): subscribes to Redis channels and emits via Socket.IO; key file main.py contains Redis listener and room-emitting logic.
- Matchmaking Service (be/services/matchMakingServices): ELO-aware matcher; publishes matchmaking\_updates.
- Leaderboard Service (be/services/leaderBoardServices): consumes domain\_events for leaderboard calculations and persistence.
- User Profile Service (be/services/userProfileServices): Postgres-backed profiles and stat sync logic listening on domain events.

## 7. Data Model (High-level)

- Game (Postgres): id (UUID), player1\_id, player2\_id, status (waiting|active|completed), started\_at, finished\_at, final\_board\_state, winner\_id, winner\_reason, metadata/settings.
- UserProfile (Postgres): user\_id (UUID), display\_name, first\_name, last\_name, elo, total\_wins, total\_losses, win\_rate, avatar\_url, current\_rank, last\_season\_rank.
- Runtime Redis State (per-game): board array, current\_turn, remaining\_time, status, winner\_id, move\_history, settings.

## 8. API Contracts & Endpoints (Representative)

Below are the high-level REST endpoints and event contracts observed in the codebase. These should be formalized into an API spec (OpenAPI) as next step.

- Game Service (REST):
  - POST /games - create a new game. Request: { player1\_id, player2\_id?, settings? }  
Response: game metadata (DB id).
  - GET /games/<game\_id> - read game state (tries Redis first, falls back to DB).
  - POST /games/<game\_id>/move - apply a move. Request: { user\_id, position }  
Response: new state or error.
  - GET /games/active/<user\_id> - list active games for user.
  - GET /games/recent/<user\_id> - list recent completed games for user.
- WebSocket Pub/Sub Events:
  - Channel game\_updates: messages { event, data, room } where room is typically game\_<game\_id> or a user id.
  - Channel domain\_events: domain-level messages { event\_type, payload } such as GAME\_COMPLETED used by stats/leaderboard services.

## 9. Distributed Systems & Scalability Strategy

Current design strengths

- Decoupled services via Redis Pub/Sub allow independent scaling and failure isolation.
- Runtime state in Redis supports fast atomic operations and low-latency gameplay.

Recommendations

- Use Redis Cluster for scale and HA. Migrate logical DBs (used in .env.example) to distinct clusters/namespaces if needed.
- Gateway scaling: run multiple gateway instances; rely on Socket.IO message\_queue backed by Redis for cross-instance emits.
- Use Redis Lua scripts or MULTI/EXEC to enforce atomic move application and prevent race conditions.
- Add a reconciliation background job that ensures completed games in Redis are persisted to Postgres, idempotently.

## 10. Cybersecurity & Compliance

Authentication & Authorization

- Requirement: JWT tokens in cookies (per user notes). Verify current authServices behavior:
  - Ensure Set-Cookie has HttpOnly, Secure (in prod), SameSite=Strict/Lax per front-end cross-site needs.

- Access tokens vs refresh tokens: use short-lived access token (e.g., 15m) and longer refresh token stored in a secure cookie; refresh via a protected endpoint.
- For service-to-service auth, use mTLS or signed service tokens (JWT) with limited scopes.

## Threat analysis (high level)

- Token theft via XSS: mitigate by storing JWTs in HttpOnly cookies and enforcing CSP and input sanitization in frontend.
- CSRF: if using cookies for auth, enforce CSRF tokens or SameSite cookie policies and verify origin headers at backend.
- Redis exposure: Redis must be protected (no public access), use ACLs, AUTH, and network isolation (VPC). Do not expose Redis to internet.
- Event spoofing: validate domain event schema and sign critical domain events (or add HMAC) to avoid unauthorized publish.
- Injection / SQLi: use ORM parameterized queries (SQLAlchemy used) and validate UUIDs (code uses uuid.UUID()).
- Privilege escalation across services: enforce per-service least privilege in DB and Redis.

## Data protection & compliance

- PII in user profiles (names, email): store hashed/limited PII; follow GDPR if applicable (data erasure flows, user consent).
- Transport and at-rest encryption:
  - Transport: TLS for HTTP and WebSocket (wss://), and TLS between services when crossing network boundaries.
  - Redis/Postgres: enable TLS and access controls in production.
- Logging: avoid logging sensitive tokens or PII in plaintext. Sanitize logs.

## Concrete actionable security items

- Ensure Set-Cookie includes HttpOnly, Secure (prod), and SameSite attributes.
- Add CSRF middleware or double-submit cookie pattern for cookie-based auth flows.
- Harden Redis (bind to private interface, require AUTH, use ACLs).
- Validate and schema-check all incoming Pub/Sub events before processing.
- Add rate limiting for login endpoints (RATELIMIT\_STORAGE\_URL present in .env.example).

# 11. Testing Strategy

- Unit tests: pure functions (game rule engine, ELO calculations), and small Flask route tests using Flask test client.
- Integration tests: local stack tests that spin up Redis & Postgres (or use testcontainers) and validate the full flow (matchmaking -> game start -> moves -> game\_over -> domain event consumption).



- End-to-end tests: scripted browser tests (Playwright) to validate lobby, connection, gameplay, timer behavior and disconnection recovery.
- Load testing: k6 or locust for socket connection ramp-up and Redis op throughput.

## 12. Implementation Roadmap (Phased timeline)

Phase 0 - Immediate fixes (1 week):

- Reproduce and fix lobby static-name bug (frontend -> API payload mismatch).
- Inspect fe matching-lobby component; confirm it consumes real match payloads from `matchmaking_updates` or `game_start`.
- Fix gameplay timer & end-of-game detection (critical).
- Validate `gameServices` move evaluation code in `state/` and `timeout_manager.py`.
- Ensure timer value propagates from settings into Redis state; frontend reads `remaining_time` from game state.
- Ensure `apply_move` computes completed status and `publish_game_update('game_over')` is triggered.
- Add integration test harness to simulate two-player match flows using local Redis.

Phase 1 - Stabilize & tests (2–3 weeks):

- Add unit + integration tests for: - Redis state transitions (`create_game`, `apply_move`, game completion).
- WebSocket Gateway event routing (subscribe -> emit) using a test Redis instance.
- Harden authentication flows: verify cookie-based JWT, refresh logic, and CSRF protection.

Phase 2 - Observability & reliability (2–3 weeks):

- Add metrics (Prometheus) on socket connections, Redis pubsub throughput, move application latency, matchmaking queue length.
- Add health checks & readiness probes for containers. Expand `verify_all_services.py` into per-service `verify_*.py` with endpoint checks.

Phase 3 - Scalability & Productionization (4–8 weeks):

- Prepare deployment manifests (Kubernetes Helm charts) or production-grade Docker Compose with HA patterns.
- Migrate Redis to clustered setup, enable TLS, and configure Redis persistence/backups.
- Add CI/CD pipelines to run tests and build images.

## 13. Risk Assessment

| Risk                                | Probability | Impact | Mitigation Strategy                                          |
|-------------------------------------|-------------|--------|--------------------------------------------------------------|
| Game state divergence (Redis vs DB) | Medium      | High   | Treat Redis as source of truth for live games; ensure atomic |

| Risk                                  | Probability | Impact      | Mitigation Strategy                                                                                                    |
|---------------------------------------|-------------|-------------|------------------------------------------------------------------------------------------------------------------------|
|                                       |             |             | write-back and reconciler for DB persistence; add idempotency keys.                                                    |
| Event schema mismatch across services | High        | Medium-High | Enforce JSON schema validation for pub/sub messages; include version field; update consumers/producers simultaneously. |
| Auth token theft / CSRF               | Medium      | High        | Use HttpOnly Secure cookies, enforce CSRF tokens or SameSite cookie + origin checks, rotate keys, short token TTLs.    |
| Redis availability (single point)     | Medium      | High        | Use Redis Cluster with replicas, automatic failover, backup/restore plan, and monitoring/alerts.                       |
| Improperly scaled WebSocket gateway   | Medium      | High        | Run multiple gateway instances with Redis message_queue and load balancing; stress-test with concurrency targets.      |
| Sensitive data leakage in logs        | Low         | Medium      | Audit logs, redact PII, and ensure logging config excludes tokens.                                                     |
| Race conditions on move application   | Medium      | High        | Use Redis transactions or Lua scripts for atomic move application and state transitions.                               |

## 14. Operational Runbook

- Local dev run (per-service):
  - Copy `be/.env.example` → `.env` and set `REDIS_URL`, `DATABASE_URL`, `FRONTEND_URL`.

- Create venv, install requirements: `pip install -r requirements.txt` inside the service folder.
  - Run: `python main.py` (services use `Config.PORT`).
- Quick system check: `python be/verify_all_services.py`.
- Health checks: add `/health` and `/ready` endpoints to each service if missing.
- Backups & restore:
  - Postgres: nightly logical backups + point-in-time recovery plan.
  - Redis: RDB/AOF configuration and periodic snapshot exports to cold storage.
- Incident response:
  - On Redis failure: promote replica or failover; put gateway into degraded mode (read-only) and surface errors to clients.
  - On DB failure: keep runtime games in Redis, queue persistence tasks for later reconciliation.

## 15. Deployment & CI/CD (Recommendations)

- CI: run unit tests, linting, and basic integration tests (against ephemeral Redis/Postgres) on PRs.
- CD: build per-service Docker images and tag by commit SHA; promote images through staging → production with database migrations gated by manual approval.
- Orchestration: prefer Kubernetes for production (Helm charts) with readiness/liveness probes. For small-scale deployments, a production-grade Docker Compose with HA proxies is acceptable.

## 16. Cost & Capacity Estimates

- To produce accurate costs, provide expected concurrent active games and average session length. Placeholder guidance:
  - Small-scale ( $\leq 500$  concurrent games): single Redis primary + replica, 2–3 web/gateway instances, managed Postgres small cluster.
  - Medium ( $> 500$ ,  $\leq 5k$ ): Redis cluster, multiple gateway replicas, autoscaling policies, Postgres with read replicas.

## 17. Conclusion

TicTacToe Arena is architected around a pragmatic set of design choices that prioritize low-latency gameplay, developer productivity, and operational simplicity. The current codebase demonstrates sound patterns — a Redis-first runtime for active games, a decoupled event bus for UI updates and domain events, and modular Flask services that map cleanly to bounded contexts (auth, game, matchmaking, profiles, leaderboards). However, the project is at a stage where operational hardening, formalized event schemas, and tighter frontend-backend contracts are necessary to move from a functional prototype toward a production-grade system.

Immediate priorities are corrective and preventative: (1) fix reproducible frontend bugs (lobby participant display, timer propagation, and deterministic game-end transitions), (2) introduce schema validation and contract tests for pub/sub messages to prevent cross-service regressions, and (3) harden authentication and cookie semantics to reduce security risk. Medium-term efforts should focus on observability (Prometheus metrics, structured logs), Redis high availability (cluster + replicas), and CI/CD pipelines that enforce testing and migration safety.

Execution of these recommendations will reduce incident blast radius, improve player experience (lower latency and fewer mismatch bugs), and provide a clear operational posture for scaling. I recommend a two-week stabilization sprint addressing the top three immediate priorities, followed by a locked release and a second sprint to add contract tests and observability. After these steps, plan a production readiness sprint (Kubernetes manifests, Redis clustering, backups, and runbooks).

## 18. References

Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.

Fette, I., & Melnikov, A. (2011). The WebSocket Protocol (RFC 6455). Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc6455>

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc7519>

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation, University of California, Irvine). <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

Elo, A. E. (1978). *The Rating of Chessplayers, Past and Present*. Arco Publishing.

OWASP Foundation. (2023). Cross-Site Request Forgery (CSRF) Prevention Cheat Sheet. [https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site\\_Request\\_Forgery\\_Prevention\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html)

Redis Ltd. (2024). Redis Documentation. <https://redis.io/docs/>

PostgreSQL Global Development Group. (2024). PostgreSQL Documentation. <https://www.postgresql.org/docs/>

Pallets Projects. (2024). Flask Documentation. <https://flask.palletsprojects.com/>

Socket.IO. (2024). Socket.IO Documentation. <https://socket.io/docs/>

Docker, Inc. (2024). Docker Documentation. <https://docs.docker.com/>

Cloud Native Computing Foundation. (2024). Kubernetes Documentation. <https://kubernetes.io/docs/>

Prometheus Authors. (2023). Prometheus: Monitoring System and Time Series Database.  
<https://prometheus.io/docs/introduction/overview/>

Richardson, C. (2018). Microservices Patterns: With examples in Java. Manning Publications.

Beyer, B., Jones, C., Petoff, J., & Murphy, N. (2016). Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media.

## 19. Glossary

This glossary explains key terms, patterns, and components used in the TicTacToe Arena project. Each entry includes a concise definition, why it matters for this codebase, and where to look for examples in the repository.

| Term                        | Definition                                                                                                                    | Why it Matters                                                            | Example / Location                                                                                                                                                             |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ELO                         | A relative skill-rating system that assigns players numeric ratings which change after matches.                               | Drives matchmaking and ranking; used to find balanced opponents.          | See <code>be/.env.example (DEFAULT_ELO_RATING)</code> and matchmaking logic in <code>be/services/matchMakingServices</code> .                                                  |
| Pub/Sub (Publish/Subscribe) | Messaging pattern where publishers emit messages to channels and subscribers receive messages for channels they subscribe to. | Decouples producers and consumers across services; used as the event bus. | Redis Pub/Sub: <code>be/services/websocketGateway/main.py</code> and <code>gameServices/routes.py</code> publish to <code>game_updates</code> and <code>domain_events</code> . |
| Room (Socket.IO)            | Logical grouping in Socket.IO to target events to specific clients (e.g., <code>game_&lt;game_id&gt;</code> or a user id).    | Ensures events reach only relevant players or dashboards.                 | <code>publish_game_update</code> uses <code>room</code> field in messages published by game service.                                                                           |
| Redis (runtime state)       | In-memory data store used for low-latency state and Pub/Sub channels.                                                         | Stores active game boards and supports fast atomic updates.               | <code>be/services/gameServices/state/</code> contains helpers like <code>create_game</code> , <code>apply_move</code> .                                                        |
| Redis Cluster               | A production-grade, sharded Redis setup providing high availability and scaling.                                              | Required for HA and scale to avoid Redis single-point-of-failure.         | Recommendation in report; production migration step.                                                                                                                           |
| WebSocket                   | Full-duplex                                                                                                                   | Enables real-time                                                         | Implemented with                                                                                                                                                               |

| Term                                 | Definition                                                                                                             | Why it Matters                                                                               | Example / Location                                                                                                  |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
|                                      | communication channel over a single TCP connection (wss/written as WebSocket).                                         | game updates and low-latency interactions.                                                   | Socket.IO in <code>be/services/websocketGateway</code> .                                                            |
| Socket.IO                            | A library that provides real-time event-based communication with fallbacks and additional features over raw WebSocket. | Simplifies rooms, reconnection, and message routing for clients and servers.                 | Gateway uses <code>flask_socketio</code> configured with Redis <code>message_queue</code> in <code>main.py</code> . |
| JWT (JSON Web Token)                 | A compact, URL-safe means of representing claims to be transferred between two parties.                                | Standard token format for authentication/authorization across services.                      | Auth service and front-end should use JWTs; see <code>be/.env.example</code> and auth service code.                 |
| CSRF (Cross-Site Request Forgery)    | Attack that forces a user to execute unwanted actions in a web application where they're authenticated.                | Cookie-based auth flows must mitigate CSRF.                                                  | Use CSRF tokens or SameSite cookie settings; noted in Security section.                                             |
| CORS (Cross-Origin Resource Sharing) | Browser mechanism that allows controlled access to resources located outside a given domain.                           | Frontend origins are whitelisted for Socket and REST calls; misconfiguration blocks clients. | <code>be/services/*/main.py</code> configures CORS using <code>env FRONTEND_URL</code> .                            |
| PostgreSQL                           | Relational database used for durable metadata like finished games and user profiles.                                   | Persistent store for audit, leaderboards, and historical data.                               | DB models under <code>be/services/*/db/</code> and migrations folders.                                              |
| Docker & docker-compose              | Containerization and local multi-container orchestration tool.                                                         | Reproducible local dev and simplified deployments.                                           | Each service has Dockerfile and many have <code>docker-compose.yml</code> under <code>be/services/*</code> .        |
| Kubernetes                           | Container orchestration platform for production deployments with autoscaling and HA.                                   | Recommended for production for orchestrating multiple services, auto-healing, and scaling.   | Suggested in roadmap; manifests not present yet.                                                                    |
| Flask Blueprints                     | Flask pattern for                                                                                                      | Keeps service code                                                                           | <code>routes.py</code> in each                                                                                      |

| Term                        | Definition                                                                                               | Why it Matters                                                                             | Example / Location                                                            |
|-----------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
|                             | modularizing routes and handlers into reusable components.                                               | organized and consistent.                                                                  | service registers a blueprint in main.py.                                     |
| Domain Event                | A semantic event representing a significant business occurrence (e.g., GAME_COMPLETED).                  | Used to trigger cross-service side-effects like stats updates and leaderboard adjustments. | gameServices publishes GAME_COMPLETED to domain_events channel.               |
| Reconciliation (write-back) | Background process that ensures durable persistence (DB) reflects runtime state (Redis).                 | Guarantees eventual consistency and handles missed writes during failures.                 | Recommended pattern in report; implement as background worker or Celery task. |
| TTL (Time To Live)          | Expiration time for ephemeral data (seconds).                                                            | Used for move timers, session expiry, and Redis key lifecycle.                             | be/.env.example contains MOVE_TIME_LIMIT and other TTLs.                      |
| Timeout Manager             | Background component that enforces time-based rules (e.g., move timeout).                                | Ensures games progress and handles forfeits/timeouts.                                      | be/services/gameServices/timeout_manager.py.                                  |
| Atomic Operation            | Operation that completes fully or not at all; often implemented using Redis transactions or Lua scripts. | Prevents race conditions when multiple clients attempt conflicting updates.                | Use Redis MULTI/EXEC or Lua scripts for apply_move operations.                |
| Lua Script (Redis)          | Server-side script executed atomically inside Redis.                                                     | Ideal for complex atomic logic (move validation + state update).                           | Recommended for apply_move to prevent race conditions.                        |
| Idempotency                 | Property that repeating the same operation has the same effect as doing it once.                         | Important for safe retries of REST calls and background tasks.                             | Use idempotency keys for move submission and DB writes.                       |
| Rate Limiting               | Policy to limit requests from a client over time.                                                        | Protects endpoints (login, matchmaking) from abuse and brute-force attacks.                | Configure using RATELIMIT_STORAGE_URL pointing to Redis (see .env.example).   |

| Term          | Definition                                                                              | Why it Matters                                                      | Example / Location                                                                           |
|---------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Observability | Collection of metrics, logs, and traces to understand system behavior.                  | Essential for debugging latency, errors, and capacity planning.     | Add Prometheus metrics + structured logs (recommendation in report).                         |
| Contract Test | Tests that validate producers and consumers of events or APIs follow the agreed schema. | Prevents cross-service regressions caused by event/payload changes. | Add JSON schema validation for Pub/Sub messages and contract tests for gateway and services. |
| Leaderboard   | Ranked list of players computed from match outcomes and ELO adjustments.                | Primary user-facing metric for competition and retention.           | be/services/leaderBoard<br>Services consumes domain_events to update ranks.                  |
| Matchmaking   | Algorithm and service that pairs players for games based on ELO and configuration.      | Core UX: fairness and wait time tradeoffs.                          | be/services/matchMakin<br>gServices and settings in .env.example.                            |



## 20. Appendices

Auth Service:

Config.py

```
1 class Config:
2     # Security
3     SECRET_KEY = os.getenv('SECRET_KEY', 'dev_secret_key_change_in_production')
4     JWT_SECRET_KEY = os.getenv('JWT_SECRET_KEY', 'jwt_secret_key_change_in_production')
5
6     # Database
7     uri = os.getenv('DATABASE_URL', 'sqlite:///dev.db')
8     if uri.startswith("postgres://"):
9         uri = uri.replace("postgres://", "postgresql://", 1)
10    SQLALCHEMY_DATABASE_URI = uri
11    SQLALCHEMY_TRACK_MODIFICATIONS = False
12
13    # JWT Configuration
14    JWT_TOKEN_LOCATION = ['cookies']
15    JWT_ACCESS_COOKIE_PATH = '/'
16    JWT_REFRESH_COOKIE_PATH = '/auth/refresh'
17    JWT_COOKIE_CSRF_PROTECT = True # Enable CSRF protection
18    JWT_ACCESS_TOKEN_EXPIRES = timedelta(minutes=15)
19    JWT_REFRESH_TOKEN_EXPIRES = timedelta(days=7)
20
21    # Cookie Security (Production vs Dev)
22    JWT_COOKIE_SECURE = os.getenv('FLASK_ENV') == 'production' # Only send over HTTPS in prod
23    JWT_COOKIE_SAMESITE = 'Lax' # Or 'Strict'
24
25    # Supabase
26    SUPABASE_URL = os.getenv('SUPABASE_URL')
27    SUPABASE_KEY = os.getenv('SUPABASE_KEY')
28
29    # Frontend URL for CORS
30    FRONTEND_URL = os.getenv('FRONTEND_URL', 'http://localhost:3000')
31
32    class DevelopmentConfig(Config):
33        DEBUG = True
34
35    class ProductionConfig(Config):
36        DEBUG = False
37
38    config = {
39        'development': DevelopmentConfig,
40        'production': ProductionConfig,
41        'default': DevelopmentConfig
42    }
```

## Register Route

```
1 @auth_bp.route('/register', methods=['POST'])
2 def register():
3     data = request.get_json()
4     email = data.get('email')
5     password = data.get('password')
6     # Do not log raw password
7     username = data.get('username')
8
9     current_app.logger.info(f"Attempting registration for email: {email}, username: {username}")
10
11     if not email or not password:
12         current_app.logger.warning("Registration failed: Missing email or password")
13         return jsonify({"message": "Email and password are required"}), 400
14
15     try:
16         # 1. Sign up with Supabase
17         # We store username in 'data' metadata so Supabase saves it
18         # redirects to frontend verify-email page
19         email_redirect_to = f"{FRONTEND_URL}/verify-email"
20
21         res = supabase.auth.sign_up({
22             "email": email,
23             "password": password,
24             "options": {
25                 "email_redirect_to": email_redirect_to,
26                 "data": {
27                     "username": username
28                 }
29             }
30         })
31
32         # 2. Sync to Local Database
33         if res.user:
34             new_user = User(
35                 id=res.user.id,
36                 email=res.user.email,
37                 username=username,
38                 password=password # Saving password locally as requested
39             )
40             db.session.add(new_user)
41             db.session.commit()
42             current_app.logger.info(f"User synced to local DB: {res.user.id}")
43
44             # 3. Create user profile in User Profile Service
45             create_user_profile(res.user.id, email, username)
46
47             # 4. Supabase sends the confirmation email automatically
48             current_app.logger.info(f"Registration successful for {email}")
49             return jsonify({
50                 "message": "Registration successful. Please check your email to verify your account.",
51                 "user_id": res.user.id if res.user else None
52             }), 201
53
54     except AuthApiError as e:
55         return jsonify({"message": e.message}), 400
56     except Exception as e:
57         return jsonify({"message": str(e)}), 500
```

## Game Service:

```
1 def get_redis_client():
2     return redis.from_url(current_app.config['REDIS_URL'])
3
4 def get_event_bus_client():
5     return redis.from_url(current_app.config['EVENT_BUS_REDIS_URL'])
6
7 def publish_game_update(game_id, event_type, data):
8     """
9     Publish event to Redis Pub/Sub for WebSocket Gateway (Event Bus).
10    """
11    try:
12        r = get_event_bus_client()
13        message = {
14            'event': event_type,
15            'data': data,
16            'room': f"game_{game_id}"
17        }
18        r.publish('game_updates', json.dumps(message))
19    except Exception as e:
20        current_app.logger.error(f"Failed to publish game update: {e}")
21
22 def publish_user_update(user_id, event_type, data):
23     """
24     Publish event to specific user's room via Redis Pub/Sub.
25     Target room: user_id (as handled in WebSocket Gateway)
26     """
27    try:
28        r = get_event_bus_client()
29        message = {
30            'event': event_type,
31            'data': data,
32            'room': str(user_id) # Publish to user's personal room
33        }
34        # Publish to 'game_updates' channel which Gateway listens to
35        r.publish('game_updates', json.dumps(message))
36    except Exception as e:
37        current_app.logger.error(f"Failed to publish user update: {e}")
```

```

1 @game_bp.route('/games', methods=['POST'])
2 def create_new_game():
3     data = request.json
4     player1_id = data.get('player1_id')
5     player2_id = data.get('player2_id') # Optional, if matchmaking or invite
6     settings = data.get('settings', {}) # Extract settings
7
8     if not player1_id:
9         return jsonify({'error': 'player1_id required'}), 400
10
11     # 1. Create DB Record
12     new_game = Game(
13         player1_id=uuid.UUID(player1_id),
14         player2_id=uuid.UUID(player2_id) if player2_id else None,
15         status='active' if player2_id else 'waiting',
16         started_at=datetime.utcnow() if player2_id else None
17     )
18     db.session.add(new_game)
19     db.session.commit()
20
21     # 2. Initialize Redis State
22     game_id = str(new_game.id)
23     redis_state = create_game(game_id, player1_id, player2_id, settings)
24
25     # 3. Notify
26     if player2_id:
27         publish_game_update(game_id, 'game_start', redis_state)
28
29         # Notify both users to update their dashboard (active games list changed)
30         publish_user_update(player1_id, 'dashboard_update', {'type': 'game_started', 'game_id': game_id})
31         publish_user_update(player2_id, 'dashboard_update', {'type': 'game_started', 'game_id': game_id})
32
33     return jsonify(new_game.to_dict()), 201

```

```

1 @game_bp.route('/games/<game_id>', methods=['GET'])
2 def get_game_state(game_id):
3     # Try Redis first
4     state = get_state(game_id)
5     if state:
6         return jsonify(state)
7
8     # Fallback to DB (historical or cache miss)
9     game = Game.query.get(game_id)
10    if not game:
11        return jsonify({'error': 'Game not found'}), 404
12    return jsonify(game.to_dict()) # DB only has metadata, not board if not finished

```



```
1 @game_bp.route('/games/active/<user_id>', methods=['GET'])
2 def get_active_games(user_id):
3     try:
4         uid = uuid.UUID(user_id)
5         games = Game.query.filter(
6             ((Game.player1_id == uid) | (Game.player2_id == uid)),
7             Game.status == 'active'
8         ).order_by(Game.started_at.desc()).all()
9         return jsonify([g.to_dict() for g in games]), 200
10    except ValueError:
11        return jsonify({'error': 'Invalid user_id'}), 400
12
13 @game_bp.route('/games/recent/<user_id>', methods=['GET'])
14 def get_recent_games(user_id):
15     try:
16         uid = uuid.UUID(user_id)
17         limit = request.args.get('limit', 5, type=int)
18         offset = request.args.get('offset', 0, type=int)
19
20         games = Game.query.filter(
21             ((Game.player1_id == uid) | (Game.player2_id == uid)),
22             Game.status == 'completed'
23         ).order_by(Game.finished_at.desc()).offset(offset).limit(limit).all()
24         return jsonify([g.to_dict() for g in games]), 200
25    except ValueError:
26        return jsonify({'error': 'Invalid user_id'}), 400
```



```
1 import os
2 from dotenv import load_dotenv
3 load_dotenv()
4 class Config:
5     SECRET_KEY = os.getenv('SECRET_KEY', 'dev_game_secret')
6     SQLALCHEMY_DATABASE_URI = os.getenv('DATABASE_URL', 'postgresql://game_user:game_pass@localhost:5434/game_db')
7     SQLALCHEMY_TRACK_MODIFICATIONS = False
8     REDIS_URL = os.getenv('REDIS_URL', 'redis://localhost:6380/0')
9     EVENT_BUS_REDIS_URL = os.getenv('EVENT_BUS_REDIS_URL', 'redis://localhost:6382/0')
10    WS_GATEWAY_URL = os.getenv('WS_GATEWAY_URL', 'http://localhost:5005')
11    USER_PROFILE_SERVICE_URL = os.getenv('USER_PROFILE_SERVICE_URL', 'http://localhost:5001')
12    INTERNAL_API_KEY = os.getenv('INTERNAL_API_KEY', 'dev_internal_key')
13    PORT = int(os.getenv('PORT', 5002))
```

## Leaderboard Service:

```
1 @leaderboard_bp.route('/leaderboard', methods=['GET'])
2 def get_leaderboard():
3     limit = request.args.get('limit', 50, type=int)
4     offset = request.args.get('offset', 0, type=int)
5
6     r = get_redis()
7
8     # Get top users (high to low)
9     # zrevrange returns list of (member, score) if withscores=True
10    raw_data = r.zrevrange('leaderboard_global', offset, offset + limit - 1, withscores=True)
11
12    results = []
13    rank = offset + 1
14
15    for user_id_bytes, score in raw_data:
16        user_id = user_id_bytes.decode('utf-8')
17
18        # Fetch metadata
19        username = r.hget(f"user:{user_id}", "username")
20        username = username.decode('utf-8') if username else "Unknown"
21
22        results.append({
23            "rank": rank,
24            "user_id": user_id,
25            "username": username,
26            "elo": int(score)
27        })
28        rank += 1
29
30    return jsonify(results), 200
31
32 @leaderboard_bp.route('/leaderboard/<user_id>', methods=['GET'])
33 def get_user_rank(user_id):
34     r = get_redis()
35
36     # Get rank (0-based)
37     rank_idx = r.zrevrank('leaderboard_global', user_id)
38     score = r.zscore('leaderboard_global', user_id)
39
40     if rank_idx is None:
41         return jsonify({"message": "User not ranked"}), 404
42
43     username = r.hget(f"user:{user_id}", "username")
44     username = username.decode('utf-8') if username else "Unknown"
45
46     return jsonify({
47         "rank": rank_idx + 1,
48         "user_id": user_id,
49         "username": username,
50         "elo": int(score)
51     }), 200
```

## Matchmaking Service:

```
1 @matchmaking_bp.route('/queue/join', methods=['POST'])
2 def join_queue():
3     data = request.json
4     user_id = data.get('user_id')
5     elo = data.get('elo')
6     game_speed = data.get('game_speed', 'standard')
7     min_elo = data.get('min_elo', 0)
8     max_elo = data.get('max_elo', 3000)
9
10    if not user_id or elo is None:
11        return jsonify({'error': 'Missing user_id or elo'}), 400
12
13    # 0. Check if user is already in an active game
14    try:
15        from config import Config
16        import requests
17        # Call Game Service
18        game_url = f"{Config.GAME_SERVICE_URL}/games/active/{user_id}"
19        response = requests.get(game_url, timeout=2)
20        if response.status_code == 200:
21            active_games = response.json()
22            if active_games and len(active_games) > 0:
23                return jsonify({'error': 'User already in an active game', 'code': 'ACTIVE_GAME'}), 400
24    except Exception as e:
25        # Log error but maybe allow queueing if game service is down?
26        # For strictness, we might want to fail, but for now let's log.
27        # Strict mode:
28        print(f"Failed to check active games: {e}")
29        # return jsonify({'error': 'Failed to verify game status'}), 500
30        pass
31
32    # check if user already in queue
33    existing = MatchQueue.query.filter_by(user_id=user_id).first()
34    if existing:
35        existing.elo = elo # Update elo if changed
36        existing.game_speed = game_speed
37        existing.min_elo = min_elo
38        existing.max_elo = max_elo
39        existing.joined_at = db.func.now() # Refresh timestamp
40    else:
41        new_entry = MatchQueue(
42            user_id=user_id,
43            elo=elo,
44            game_speed=game_speed,
45            min_elo=min_elo,
46            max_elo=max_elo
47        )
48        db.session.add(new_entry)
49
50    try:
51        db.session.commit()
52        return jsonify({'message': 'Joined queue', 'status': 'queued'}), 200
53    except Exception as e:
54        db.session.rollback()
55        return jsonify({'error': str(e)}), 500
56
57 @matchmaking_bp.route('/queue/leave', methods=['POST'])
58 def leave_queue():
59     data = request.json
60     user_id = data.get('user_id')
61
62     if not user_id:
63         return jsonify({'error': 'Missing user_id'}), 400
64
65     entry = MatchQueue.query.filter_by(user_id=user_id).first()
66     if entry:
67         db.session.delete(entry)
68         db.session.commit()
69         return jsonify({'message': 'Left queue', 'status': 'removed'}), 200
70
71     return jsonify({'message': 'User not in queue'}), 404
```